UNIVERSITATEA POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI

Proiect: Sistem Automatizat de Colectare a Datelor și Aplicație Web pentru Repertoriul Teatral

Studenți: Vasile Adina – Ioana, Rusu Ioana - Sorina
Grupa: 443B
Profesor îndrumător: Prof. Dr. Bogdan Florea

Bucuresti 2026

Cuprins

1. INTRODUCERE ȘI OBIECTIVELE PROIECTULUI 3
2. TEHNOLOGII ȘI MEDII DE DEZVOLTARE 3
3. ARHITECTURA SISTEMULUI ȘI BAZA DE DATE 4
3.1 Schema bazei de date 4
3.2 Idempotența operațiunilor SQL 5
4. IMPLEMENTAREA MODULULUI DE WEB SCRAPING (BACKEND) 7
4.1 Evazivitatea și ocolirea protecțiilor Anti-Bot 7
4.2 Navigarea și extragerea structurală (DOM Parsing) 8
5. IMPLEMENTAREA INTERFEȚEI GRAFICE (FRONTEND) 11
5.1. Sistemul de rutare și managementul stării 11
5.2. Conexiunea optimizată (Caching) 12
5.3. Interfața dinamică 12
6. INTEGRAREA INTELIGENȚEI ARTIFICIALE ÎN DEZVOLTARE 14
7. Concluzii 14
8. Bibliografie 16
Anexe 17

1. INTRODUCERE ȘI OBIECTIVELE PROIECTULUI

În contextul digitalizării masive a instituțiilor culturale, accesul rapid și centralizat la informațiile despre evenimentele artistice reprezintă o necesitate. Proiectul de față propune o soluție software completă (end-to-end) pentru agregarea, structurarea și prezentarea programului teatral.

Obiectivele principale ale proiectului sunt:

- **Automatizarea colectării de date (Web Scraping):** Extragerea automată a informațiilor (titlu, distribuție, echipă tehnică, date calendaristice, afișe) de pe platforma **Teatrului Mic**.
- **Normalizarea și Stocarea Persistentă:** Proiectarea unei baze de date relaționale (MySQL) capabile să gestioneze entități complexe (spectacole și reprezentații) și să evite duplicarea datelor.
- **Dezvoltarea unei Interfețe Web:** Crearea unei aplicații web interactive și responsive care să permită utilizatorului final explorarea repertoriului și a calendarului.
- **Toleranța la erori și mentenabilitatea:** Implementarea de mecanisme anti-bot și algoritmi de parsare reziliențiși la modificările structurale ale codului sursă HTML.

2. TEHNOLOGII ȘI MEDII DE DEZVOLTARE

În dezvoltarea web modernă, accesul la fluxuri de date curate reprezintă o provocare constantă, mai ales când vine vorba despre instituții culturale precum teatrele. În mod ideal, o platformă oferă un **API** dedicat sau un flux **RSS** prin care alte aplicații pot consuma datele direct în formate structurate. Totuși, majoritatea site-urilor publică repertoriul și programul exclusiv sub formă de pagini **HTML**, gândite doar pentru afișarea în browser. Când nu există o cale directă de integrare, colectarea automată a informațiilor devine singura opțiune viabilă. [1]

Această colectare automată se bazează pe trei mecanisme complementare: **crawling**, **parsing** și **scraping**. Un **web crawler** (sau *spider*) are rolul de a parcurge sistematic internetul. El pornește de la o pagină de start, urmărește linkurile și găsește adresele relevante, construind practic harta de navigare. Odată descărcată pagina HTML, intervine etapa de **parsing**, adică analiza sintactică prin care codul brut este transformat într-un arbore structurat de obiecte numit **DOM** (Document Object Model). Peste această structură acționează procesul de **web scraping**, care extrage doar valorile care contează — titlul spectacolului, distribuția, sala sau data reprezentației — și le pregătește pentru salvare. [2][3]

Pentru a rula un astfel de proces în mod etic și eficient, este esențială respectarea fișierului **robots.txt**, standardul prin care administratorul unui site dictează ce zone pot fi vizitate de roboți și ce rute sunt restricționate. În același timp, ritmul cererilor HTTP trebuie ținut sub control prin pauze regulate, pentru a nu suprasolicita infrastructura serverului. [4]

La nivel de implementare, un ecosistem bazat pe **Python** oferă toate instrumentele necesare pentru un flux complet. Biblioteca **HTTPX** descarcă rapid paginile web și oferă un

control precis asupra headerelor de rețea, în timp ce **BeautifulSoup4** se ocupă de parsarea eficientă a arborelui DOM și localizarea elementelor dorite. Textele colectate sunt filtrate adesea cu **expresii regulate (Regex)** pentru a elimina spațiile inutile, tagurile reziduale sau pentru a scoate linkuri ascunse în stiluri CSS. [5]

Datele astfel curățate sunt stocate persistent într-o bază de date relațională **MySQL**, care păstrează integritatea legăturilor dintre spectacole și reprezentații prin chei externe și suportă câmpuri de tip JSON. În final, interfața utilizatorului este construită prin **Streamlit**, un framework care permite afișarea rapidă a programului și a detaliilor spectacolelor într-o aplicație web interactivă.

3. ARHITECTURA SISTEMULUI ȘI BAZA DE DATE

Sistemul este împărțit în două module decuplate: modulul de backend (crawler-ul) și modulul de frontend (aplicația web). Această separare arhitecturală permite rularea independentă a crawler-ului fără a afecta disponibilitatea interfeței web.

3.1 Schema bazei de date

Pentru persistența și structurarea datelor extrase, s-a utilizat un sistem relațional MySQL (motor de stocare `InnoDB`) configurat complet pe setul de caractere `utf8mb4` cu colaționarea `utf8mb4_unicode_ci`. Această configurare este esențială pentru a asigura stocarea corectă a caracterelor speciale și a diacriticelor din limba română (ș, ț, â, î, ă).

Baza de date este organizată în jurul a două tabele principale legate relațional:

A. Tabelul `shows` (Spectacole)

Stochează metadatele generale ale fiecărei producții teatrale:

```
CREATE TABLE IF NOT EXISTS shows (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT NULL,
    cast_json JSON NULL,
    team_json JSON NULL,
    poster_storage ENUM('file', 'base64') NULL,
    poster_path VARCHAR(1024) NULL,
    poster_hash CHAR(64) NULL,
    source_url VARCHAR(2048) NOT NULL,
    last_seen_at TIMESTAMP NULL,
    ...
    INDEX idx_shows_title (title)
);
```

- **Câmpurile native** `JSON (cast_json, team_json)`: permit salvarea flexibilă a listei de actori și a membrilor echipei artistice (regie, scenografie etc.) fără a fi necesară o fragmentare și o normalizare excesivă în multiple tabele asociative.

- **Gestiunea afișelor locale:** coloana `poster_storage` marchează modalitatea de stocare aleasă (`file`), `poster_path` reține calea relativă către fișierul descărcat pe disc, iar poster_hash păstrează amprenta SHA-256 a imaginii pentru a detecta rapid modificările de conținut între rulări.
- **Indecși de căutare**: `idx_shows_title` optimizează performanța interogărilor de căutare directă a spectacolelor după titlu.

B. Tabelul `performances` (Reprezentații)

Modelează instanțele calendaristice ale spectacolelor (programul efectiv):

```
CREATE TABLE IF NOT EXISTS performances (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    show_id BIGINT UNSIGNED NOT NULL,
    start_at TIMESTAMP NOT NULL,
    venue VARCHAR(255) NULL,
    source_url VARCHAR(2048) NULL,
    last_seen_at TIMESTAMP NULL,
    ...
    CONSTRAINT fk_performances_show
        FOREIGN KEY (show_id) REFERENCES shows(id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    INDEX idx_performances_show_date (show_id, start_at)
);
```

- **Integritate referențială:** cheia externă `fk_performances_show` leagă fiecare reprezentație de producția asociată din tabela `shows`. Clauza `ON DELETE CASCADE` garantează ștergerea automată a calendarului aferent dacă un spectacol este eliminat din sistem.
- Index compus `idx_performances_show_date`: optimizează căutările care filtrează simultan reprezentațiile după identificatorul spectacolului și data de desfășurare (cum ar fi găsirea următoarei reprezentații viitoare).

3.2 Idempotența operațiunilor SQL

Una dintre cerințele cheie ale crawler-ului este capacitatea de a fi rulat în mod repetat fără a crea înregistrări duplicate și actualizând informațiile care au suferit modificări pe site-ul sursă. Acest comportament poartă denumirea de **idempotență** și este implementat la nivel de bază de date prin mecanismul de *Upsert* (`INSERT ... ON DUPLICATE KEY UPDATE`).

A. Actualizarea automată a spectacolelor

La inserarea unei producții, dacă înregistrarea există deja (pe baza unei constrângeri unice sau a cheii primare), comanda SQL actualizează automat valorile curente în loc să eșueze cu o eroare de duplicare:

```
INSERT INTO shows (
    title, description, cast_json, team_json,
    poster_storage, poster_path, poster_source_url,
    poster_hash, source_url, last_seen_at
```

```
)
VALUES (%s, %s, %s, %s, 'file', %s, %s, %s, %s, NOW())
ON DUPLICATE KEY UPDATE
    description = VALUES(description),
    cast_json = VALUES(cast_json),
    team_json = VALUES(team_json),
    poster_path = VALUES(poster_path),
    last_seen_at = NOW();
```

- `VALUES(...)`: suprascrie detaliile vechi (sinopsis, distribuție, echipă sau calea afișului) cu cele mai noi date colectate din pagină.
- `last_seen_at = NOW()`: actualizează data și ora ultimei vizitări cu timestamp-ul curent. Acest marcaj permite identificarea rapidă a spectacolelor care au ieșit din repertoriul instituției (cele care nu au mai primit actualizare la rularea recentă).

B. Evitarea duplicării reprezentațiilor

Pentru calendarul reprezentațiilor se aplică un mecanism similar:

```
INSERT INTO performances (show_id, start_at, venue, source_url, last_seen_at)
VALUES (%s, %s, 'Teatrul Mic', %s, NOW())
ON DUPLICATE KEY UPDATE
    last_seen_at = NOW();
```

- Previne inserarea aceleiași reprezentații de mai multe ori la rulări succesive, limitându-se la a confirma prezența acesteia pe site prin actualizarea câmpului de audit `last_seen_at`.

C. Definirea constrângerilor de unicitate

Pentru ca mecanismul de `ON DUPLICATE KEY UPDATE` să funcționeze efectiv, a fost necesară adăugarea a două constrângeri unice explicite la nivelul tabelelor:

```
ALTER TABLE shows
ADD CONSTRAINT uq_shows_source_url UNIQUE (source_url(500));

ALTER TABLE performances
ADD CONSTRAINT uq_performance_show_time UNIQUE (show_id, start_at);
```

- **Spectacole (`uq_shows_source_url`)**: În schema inițială, singura cheie unică era id-ul autoincrementat; deoarece la inserare nu se furniza un id, baza de date genera mereu o înregistrare nouă, ignorând actualizarea. Prin setarea URL-ului paginii sursă ca identificator unic, baza de date recunoaște spectacolul deja existent și declanșează ramura de actualizare în loc să creeze dubluri.
- **Reprezentații (`uq_performance_show_time`)**: Blochează multiplicarea evenimentelor calendaristice prin forțarea unicității compuse dintre spectacol (`show_id`) și momentul începerii (`start_at`), permițând actualizarea simplă a stării acestora.

  Codul integral poate fi gasit la **Anexa 1.**

4. IMPLEMENTAREA MODULULUI DE WEB SCRAPING (BACKEND)

4.1 Evazivitatea și ocolirea protecțiilor Anti-Bot

Pentru a evita blocarea cererilor automate de către mecanismele de securitate ale serverului web, modulul emulează comportamentul unui browser legitim prin tehnica de *header spoofing*.

Definim un dicționar complet de antete HTTP care imită o sesiune de navigare pe desktop:

```
headere_browser = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
    "Accept-Language": "ro-RO,ro;q=0.9,en-US;q=0.8,en;q=0.7",
    "Referer": "https://www.google.com/"
}
```

- **User-Agent:** Indică serverului că cererea provine de la un browser Google Chrome instalat pe Windows 10, mascând clientul automat httpx.
- **Accept și Accept-Language:** Transmit formatele de documente acceptate și preferința lingvistică locală (ro-RO), conform unei accesări reale din România.
- **Referer:** Simulează proveniența utilizatorului dintr-o căutare Google, evitând declanșarea filtrelor de acces direct neautorizat.

Înainte de a iniția descărcarea paginilor, crawlerul validează permisiunile din fișierul de politici al site-ului folosind biblioteca nativă `urllib.robotparser`:

```
def permite_robots(url):
    try:
        url_robots = f"{url_baza}/robots.txt"
        raspuns = httpx.get(url_robots, headers=headere_browser, timeout=10.0)
        if raspuns.status_code != 200:
            return True
        rp = RobotFileParser()
        rp.parse(raspuns.text.splitlines())
        return rp.can_fetch(headere_browser["User-Agent"], url)
    except Exception:
        return True
```

- **RobotFileParser**: Descarcă și analizează direct fișierul `robots.txt` conform standardului RFC 9309. Metoda `can_fetch` verifică dacă URL-ul dorit poate fi accesat sau dacă este restricționat prin directiva Disallow

Efectuăm cererea HTTP către pagina principală folosind clientul httpx:

```
pagina_principala = httpx.get(f"{url_baza}/spectacole/", headers = headere_browser, follow_redirects = True)
pagina_principala.raise_for_status()
```

Pentru a respecta bunele practici și a nu supraîncărca serverul cu cereri consecutive rapide (ceea ce ar putea genera erori HTTP 429 *Too Many Requests*), s-a introdus un mecanism de *rate limiting*:

```
for link in linkuri_spectacole:
    time.sleep(1.0)
```

- **time.sleep(1.0)**: Oprește execuția timp de o secundă între descărcarea a două pagini consecutive de spectacol, garantând un flux bun de colectare a datelor.

4.2 Navigarea și extragerea structurală (DOM Parsing)

A. Descoperirea linkurilor către spectacole (Crawling)

Transformăm codul sursă descărcat într-un arbore DOM cu ajutorul parserului *html.parser* din BeautifulSoup:

```
soup = BeautifulSoup(pagina_principala.text, 'html.parser')
toate_tagurile_a = soup.find_all('a', href=True)
linkuri_spectacole = []

for tag in toate_tagurile_a:
    href = tag.get('href')
    if '/spectacol/' in href and href not in linkuri_spectacole:
        linkuri_spectacole.append(href)
```

- `find_all('a', href=True)`: Selectează toate ancorele din pagină care au definit atributul de legătură href.
- `if '/spectacol/' in href and href not in linkuri_spectacole`: Filtrează doar adresele URL care indică pagini individuale de spectacol și previne salvarea adreselor duplicate în lista de parcurs.

B. Extragerea titlului și a sinopsisului

Pentru fiecare pagină individuală, interogăm clasele CSS specifice:

```
titlu_tag = soup_spect.find('h1', class_='title')
titlu = titlu_tag.text.strip() if titlu_tag else "Fără titlu"

desc_tag = soup_spect.find('div', class_='content-info')
descriere = desc_tag.text.strip() if desc_tag else ""
```

- `find('h1', class_='title')`: Izolează nodul de titlu principal, eliminând spațiile libere perimetrice prin .strip().
- `find('div', class_='content-info')`: Extrage conținutul descriptiv (sinopsisul), returnând un șir gol în cazul în care tag-ul lipsește de pe pagină.

C. Preluarea afișului din stiluri inline prin Regex

Afișul spectacolului este extras din proprietatea CSS de fundal, verificându-se amprenta binară pentru a evita descărcările redundante:

```
afis_tag = soup_spect.find('div', class_='header-show')
url_afis = None
```

```
if afis_tag and afis_tag.get('style'):
       potrivire_url = re.search(r"url\(['\"]?(.*?)['\"]?\)", afis_tag.get('style'))
       if potrivire_url:
              url_afis = potrivire_url.group(1)

       cale_afis, hash_afis = descarca_afis(url_afis)
date_imagine = raspuns.content
hash_imagine = hashlib.sha256(date_imagine).hexdigest()
cale_fisier = os.path.join(folder_afise, f"{hash_imagine}.jpg")
if not os.path.exists(cale_fisier):
   with open(cale_fisier, "wb") as f:
       f.write(date_imagine)
```

- `afis_tag.get('style')`: Citește stilul CSS inline al containerului de header.
- `re.search(...)`: Aplică o expresie regulată pentru a extrage valoarea din interiorul funcției url(...), ignorând ghilimelele simple sau duble.
- `hashlib.sha256(date_imagine).hexdigest()`: Calculează amprenta unică a imaginii pentru detectarea modificărilor grafice.
- `if not os.path.exists(cale_fisier)`: În interiorul funcției de descărcare, imaginea este scrisă pe disc doar dacă nu a fost deja descărcată anterior, economisind spațiu și lățime de bandă.

D. Parsarea distribuției și a echipei artistice (Generare JSON)

Secțiunea de distribuție combină rolurile tehnice și personajele într-o listă text continuă:

```
container_distributie = soup_spect.find('div', class_='distributie')
distributie = []
echipa_artistica = []

if container_distributie:
    linii = [l.strip() for l in container_distributie.get_text(separator="\n").split("\n") if
len(l.strip()) > 1]
    suntem_la_actori = False
    i = 0
    while i < len(linii) - 1:
        stanga = linii[i]
        dreapta = linii[i+1]
        if "distributie" in stanga.lower() or "distribuție" in stanga.lower():
            suntem_la_actori = True
            i += 1
            continue
        if suntem_la_actori:
            distributie.append({"character": stanga, "name": dreapta})
        else:
            echipa_artistica.append({"role": stanga, "name": dreapta})
        i += 2
```

```
json_distributie = json.dumps(distributie if distributie else [{"name": "Nu a fost publicată"}], ensure_ascii=False)
json_echipa = json.dumps(echipa_artistica if echipa_artistica else [{"role": "Informație", "name": "Nepublicată"}], ensure_ascii=False)
```

- `get_text(separator='\n')`: Sparge textul containerului pe linii separate păstrând ordinea logică a elementelor.
- Parcurgerea în salturi de câte două poziții (`i += 2`): Asociază perechile de tip rol - nume sau personaj - actor în funcție de atingerea separatorului textual "distribuție".
- `ensure_ascii=False`: Garantează că diacriticele din limba română sunt stocate corect în formatul JSON fără codificări de tip unicode escape

E. Extragerea și normalizarea metadatelor spectacolului

Pentru extragerea datelor tehnice și a creditelor esențiale (autor, regizor, durată, vârstă recomandată), modulul apelează funcția specializată `extrage_metadate_spectacol(soup, echipa)`:

```
autor, regizor, durata_text, durata_min, varsta, limba = extrage_metadate_spectacol(soup_spect, echipa_artistica)
```

- **Autor**: Este căutat inițial în tag-urile cu clase dedicate (cum ar fi .by sau clase care conțin subtitle/author). Se folosește o expresie regulată (`re.sub(r"^de\s*[:\-]?\s*", "", ...)`) pentru a curăța prefixele de tip „de: ”. Dacă lipsește din aceste clase, autorul este căutat ca rezervă în lista `echipa_artistica` după roluri asociate („autor”, „text”, „dramaturgie”).
- **Autor**: Este extras din dicționarul `echipa_artistica` prin filtrarea după termenii „regie” sau „regizor”.
- **Durată:** Se extrage formatul textual (ex: "1h 45min" sau "90 min") din corpul paginii, după care este convertit într-un număr întreg reprezentând numărul total de minute (`durata_min`), facilitând operațiile ulterioare de filtrare numerică în SQL.
- **Recomandare de vârstă**: Expresiile regulate extrag marcaje precum "14+", "+16" sau "peste 12 ani", eliminând totodată fals-pozitivele nerelevante (de tip "+1").

F. Normalizarea datelor de calendar și maparea lunilor

Pentru fiecare spectacol sunt extrase datele exacte de desfășurare, făcându-se corelarea directă cu link-ul spre platforma de cumpărare a biletelor:

```
elemente_calendar = soup_spect.find_all("div", class_="left")
for element in elemente_calendar:
    date_tag = element.find("span", class_="date")
    time_tag = element.find("span", class_="time")
    rand_eveniment = element.find_parent("div", class_=re.compile(r"item|row|program", re.I)) or element.parent

    url_bilet = None
    if rand_eveniment:
```

```
        tag_bilet                          =                          rand_eveniment.find("a",
href=re.compile(r"ticketsys|bilet|ticket|event",    re.I))    or    rand_eveniment.find("a",
href=True)
        if tag_bilet and tag_bilet.get("href"):
            href_curat = tag_bilet["href"].strip()
            url_bilet     =     href_curat     if     href_curat.startswith("http")     else
f"{url_baza}{href_curat}"
```

Normalizarea temporară folosește o abordare robustă pe două niveluri:

1. **Metoda 1 (Extragere din URL-ul biletului)**: Dacă adresa include data evenimentului (ex: ...-01-11-2026-1830), se aplică regex direct pe șir:

```
m_url = re.search(r'(\d{1,2})-(\d{1,2})-(\d{4})', url_bilet)
```

2. **Metoda 2 (Extragere din text și mapare lunară)**: Dacă URL-ul nu conține data, scriptul caută formate numerice de tip `zz.ll.aaaa` în rândul evenimentului sau combină ziua din `date_tag` cu dicționarul de mapare `luni`:

```
mapare_luni = {'ian': 1, 'feb': 2, 'mar': 3, 'apr': 4, 'mai': 5, 'iun': 6,
               'iul': 7, 'aug': 8, 'sep': 9, 'oct': 10, 'noi': 11, 'dec': 12}
```

Rezolvarea schimbării de an (Stagiune teatrală trans-anuală): Pentru a preveni erorile de asociere atunci când programul conține reprezentații din lunile de început ale anului viitor, se ajustează anul calendaristic:

```
acum = datetime.now()
if acum.month >= 9 and luna < 9:
    an = acum.year + 1
```

Codul integral poate fi regasit in **Anexa 2**.

5. IMPLEMENTAREA INTERFEȚEI GRAFICE (FRONTEND)

Partea vizuală a proiectului (app.py) este dezvoltată utilizând Streamlit, oferind o experiență de tip Single Page Application (SPA).

5.1. Sistemul de rutare și managementul stării

Deși Streamlit funcționează nativ prin re-executarea scriptului de sus până jos la fiecare interacțiune, aplicația emulează rutele cerute în specificații prin intermediul parametrilor din adresa URL (`st.query_params`):

```
query_params = st.query_params
show_id = query_params.get("id")
page = query_params.get("page", "shows")
```

- `st.query_params`: Citește parametrii din bara de adrese a browserului pentru a determina starea curentă a aplicației (ecranul activ).
- `page == "shows"`: Reprezintă starea implicită, afișând catalogul general de spectacole.
- `show_id`: Identifică dacă utilizatorul a selectat o producție specifică, declanșând afișarea detaliilor.

Schimbarea ecranului se realizează programatic prin modificarea acestor parametri și forțarea reîncărcării:

```
if st.button("Vezi detalii", key=f"btn_{s['id']}"):
    st.query_params["id"] = s['id']
    st.rerun()
```

- `st.query_params["id"] = s['id']`: Setează parametrul de identificare al spectacolului în URL.
- `st.rerun()`: Re-execută scriptul imediat, permițând trecerea la vizualizarea de detaliu fără a necesita un server complex de rutare pe backend.
- Butoanele de navigare inversă resetează parametrii prin `st.query_params.clear()`, întorcând utilizatorul la vizualizarea principală.

5.2. Conexiunea optimizată (Caching)

Pentru a preveni deschiderea și închiderea repetată a conexiunilor la serverul MySQL la fiecare randare sau click al utilizatorului, s-a implementat mecanismul de memorare în cache din Streamlit:

```
@st.cache_resource
def init_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="abcd1234",
        database="theater_crawler"
    )

conn = init_connection()
```

- `@st.cache_resource`: Instruiește mediul de execuție să păstreze instanța conexiunii activă pe durata întregii sesiuni de rulare, partajând-o între re-randări succesive.
- Elimină latența generată de re-autentificarea continuă la nivelul bazei de date și reduce consumul de resurse al serverului MySQL.

Interogările sunt centralizate într-o funcție dedicată:

```
def fetch_data(query, params=None):
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, params or ())
    return cursor.fetchall()
```

- `dictionary=True`: Returnează rândurile din tabele direct sub formă de dicționare Python (asociind numele coloanei cu valoarea), simplificând inserarea lor în elementele de interfață.

5.3. Interfața dinamică

A. Grila modulară pentru repertoriu

Ecranul principal organizează spectacolele într-o grilă modulară cu trei coloane responsive:

```
cols = st.columns(3)
for i, s in enumerate(spectacole):
       with cols[i % 3]:
              st.subheader(s['title'])
              if s['poster_path'] and os.path.exists(s['poster_path']):
                   st.image(s['poster_path'], use_container_width=True)
```

- `st.columns(3)`: Împarte spațiul vizual în 3 coloane egale.
- `with cols[i % 3]`: Distribuie iterativ fiecare spectacol pe coloana corespunzătoare.
- `os.path.exists(...)`: Validează existența fișierului pe disc local înainte de randare, prevenind erorile vizuale dacă un afiș lipsește.

B. Structurarea paginii individuale a spectacolului

Când parametrul *id* este prezent, pagina se reconfigurează pentru a prezenta toate informațiile descriptive și artistice:

```
col1, col2 = st.columns([1, 2])
with col1:
       st.image(s['poster_path'], use_container_width=True)
with col2:
       st.markdown("### Descriere")
       st.write(s['description'])
```

- Împarte ecranul asimetric: o treime din lățime este alocată afișului local, iar două treimi sunt dedicate detaliilor tehnice și sinopsisului.

Distribuția și echipa artistică stocate ca JSON în MySQL sunt deserializate și afișate structurat:

```
cast = json.loads(s['cast_json']) if s['cast_json'] else []
for actor in cast:
    nume = actor.get('name', 'Necunoscut')
    personaj = actor.get('character', '')
    if personaj:
        st.markdown(f"- **{nume}** (ca *{personaj}*)")
    else:
        st.markdown(f"- {nume}")
```

- `json.loads(...)`: Convertește șirul text JSON extras din coloana cast_json într-o listă de dicționare Python.
- Formatează fiecare intrare, evidențiind numele actorului și rolul interpretat conform formatului cerut în proiect.

C. Tabelul cronologic al reprezentațiilor

```
reprezentatii = fetch_data("""
    SELECT p.start_at, p.venue, s.title
```

```
    FROM performances p
    JOIN shows s ON p.show_id = s.id
    ORDER BY p.start_at ASC
""")
st.dataframe(reprezentatii, use_container_width=True, hide_index=True)
```

- Folosește o interogare cu JOIN între *performances* și *shows* pentru a asocia fiecare reprezentație cu titlul producției.
- `st.dataframe(...)`: Afișează datele într-un tabel interactiv care permite sortarea coloanelor și adaptarea automată la lățimea ecranului `(use_container_width=True)`

Codul integral poate fi gasit la **Anexa 3.**

6. INTEGRAREA INTELIGENȚEI ARTIFICIALE ÎN DEZVOLTARE

Pe parcursul proiectului, inteligența artificială a fost utilizată doar ca un instrument de asistență rapidă, fără a înlocui logica principală a aplicației. Structura scriptului, fluxul de parcurgere a paginilor și organizarea bazei de date au fost stabilite independent, AI-ul intervenind punctual pentru verificări sau deblocarea unor pași specifici.

Un aport practic a existat în zona expresiilor regulate (Regex). Instrumentul a servit la calibrarea și testarea rapidă a tiparelor folosite pentru extragerea link-ului de afiș din stilurile CSS, a formatelor de durată, a restricțiilor de vârstă și a datelor calendaristice din linkurile de bilete, scurtând timpul alocat testării manuale.

Totodată, au fost solicitate clarificări legate de sintaxa comenzilor SQL, în special pentru clauzele de tip UPSERT (ON DUPLICATE KEY UPDATE), asigurând evitarea duplicatelor la rulări repetate. Fiecare soluție propusă a fost analizată, ajustată și integrată direct în funcție de cerințele concrete ale site-ului parcurs.

7. Concluzii

Proiectul confirmă eficiența combinării tehnicilor de web scraping cu soluțiile moderne de vizualizare interactivă, demonstrând cum poate fi dezvoltat un flux complet autonom de preluare și afișare a datelor. Arhitectura aleasă asigură o toleranță ridicată la variațiile de structură din paginile sursă, în timp ce constrângerile definite la nivelul bazei de date garantează integritatea informațiilor și elimină complet redundanțele.

Ca direcții viitoare de extindere, soluția oferă o bază solidă pentru scalare și noi funcționalități:

- **Extinderea arhitecturii multi-sursă**: adaptarea crawlerului pentru a agrega simultan date de pe site-urile mai multor teatre, transformând aplicația într-un hub cultural centralizat pentru evenimentele din București.

- **Sistem de alerte automate**: configurarea unui serviciu de notificare (de exemplu, prin e-mail) care să informeze utilizatorii în timp util atunci când se pun în vânzare reprezentații pentru spectacolele de interes.
- **Analiză avansată de conținut (NLP)**: colectarea cronicilor și a recenziilor apărute în presa culturală, urmată de aplicarea unor modele de procesare a limbajului natural pentru analiza sentimentelor și generarea unor sinteze critice automatizate.

8. Bibliografie

[1]Prof. Dr. Bogdan Florea. Crawlere și scrapere web.

[2]Wikipedia. *Web scraping*. Disponibil: https://en.wikipedia.org/wiki/Web_scraping.

[3]Wikipedia. *Web crawler*. Disponibil: https://en.wikipedia.org/wiki/Web_crawler.

[4]Wikipedia. *robots.txt*. Disponibil: https://en.wikipedia.org/wiki/Robots.txt.

[5]Wikipedia. *Beautiful Soup (HTML parser)*. Disponibil: https://en.wikipedia.org/wiki/Beautiful_Soup_(HTML_parser).

[6] Richardson, L., *Beautiful Soup 4 Documentation*, 2024. Disponibil: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[7] Streamlit Inc., *Streamlit Library Reference and Documentation*, 2024. Disponibil: https://docs.streamlit.io/

[8] Oracle, *MySQL 8.0 Reference Manual*, Oracle Corporation, 2024.. Disponibil: https://dev.mysql.com/doc/refman/8.0/en/

[9] Python Software Foundation, *Python 3.11 Documentation - Regular Expressions (re)*. Disponibil: https://docs.python.org/3/library/re.html

[10] Encode OSS, HTTPX - *A next generation HTTP client for Python*, 2024. Disponibil: https://www.python-httpx.org/

[11] Mitchell, R., *Web Scraping with Python: Collecting More Data from the Modern Web*, O'Reilly Media, 2nd Edition, 2018.

Anexe

Anexa 1

```
CREATE DATABASE IF NOT EXISTS theater_crawler
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
USE theater_crawler;
CREATE TABLE IF NOT EXISTS shows (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
author VARCHAR(255) NULL,
director VARCHAR(255) NULL,
description TEXT NULL,
genre VARCHAR(255) NULL,
language VARCHAR(255) NULL,
duration_minutes INT UNSIGNED NULL,
duration_text VARCHAR(255) NULL,
age_recommendation VARCHAR(255) NULL,
premiere_date DATE NULL,
cast_json JSON NULL,
team_json JSON NULL,
poster_storage ENUM('file', 'base64') NULL,
poster_path VARCHAR(1024) NULL,
poster_base64 LONGTEXT NULL,
poster_mime_type VARCHAR(100) NULL,
poster_source_url VARCHAR(2048) NULL,
poster_hash CHAR(64) NULL,
source_url VARCHAR(2048) NOT NULL,
last_seen_at TIMESTAMP NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
INDEX idx_shows_title (title),
INDEX idx_shows_author (author),
INDEX idx_shows_director (director),
INDEX idx_shows_premiere_date (premiere_date)
)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_unicode_ci;
CREATE TABLE IF NOT EXISTS performances (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
show_id BIGINT UNSIGNED NOT NULL,
start_at TIMESTAMP NOT NULL,
end_at TIMESTAMP NULL,
venue VARCHAR(255) NULL,
```

```
performance_type VARCHAR(100) NULL,
status VARCHAR(50) NULL,
notes TEXT NULL,
source_url VARCHAR(2048) NULL,
last_seen_at TIMESTAMP NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_performances_show
FOREIGN KEY (show_id)
REFERENCES shows(id)
ON DELETE CASCADE
ON UPDATE CASCADE,
INDEX idx_performances_show_id (show_id),
INDEX idx_performances_start_at (start_at),
INDEX idx_performances_venue (venue),
INDEX idx_performances_show_date (show_id, start_at)
)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_unicode_ci;

ALTER TABLE shows
ADD CONSTRAINT uq_shows_source_url UNIQUE (source_url(500));

ALTER TABLE performances
ADD CONSTRAINT uq_performance_show_time UNIQUE (show_id, start_at);
```

Anexa 2

```
import httpx
from bs4 import BeautifulSoup
import mysql.connector
import json
import os
import hashlib
from datetime import datetime
import re
import time
from urllib.robotparser import RobotFileParser

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="abcd1234",
    database="theater_crawler"
)
```

```
cursor = db.cursor(dictionary=True)

url_baza = "https://www.teatrulmic.ro"
folder_afise = "static/posters"
os.makedirs(folder_afise, exist_ok=True)

headere_browser = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
    "Accept-Language": "ro-RO,ro;q=0.9,en-US;q=0.8,en;q=0.7",
    "Referer": "https://www.google.com/"
}

def permite_robots(url):
    try:
        url_robots = f"{url_baza}/robots.txt"
        raspuns = httpx.get(url_robots, headers=headere_browser, timeout=10.0)
        if raspuns.status_code != 200:
            return True
        rp = RobotFileParser()
        rp.parse(raspuns.text.splitlines())
        return rp.can_fetch(headere_browser["User-Agent"], url)
    except Exception:
        return True

def descarca_afis(url_imagine):
    if not url_imagine:
        return None, None

    try:
        if not url_imagine.startswith("http"):
            url_imagine = url_baza + url_imagine

        raspuns = httpx.get(url_imagine, headers=headere_browser, timeout=10.0)
        raspuns.raise_for_status()

        date_imagine = raspuns.content
        hash_imagine = hashlib.sha256(date_imagine).hexdigest()
        cale_fisier = os.path.join(folder_afise, f"{hash_imagine}.jpg")

        if not os.path.exists(cale_fisier):
            with open(cale_fisier, "wb") as f:
                f.write(date_imagine)
```

```
        return cale_fisier, hash_imagine
    except Exception as e:
        print(f"⚠️ Eroare descărcare afiș: {e}")
        return None, None

def extrage_metadate_spectacol(soup, echipa):
    autor = None
    regizor = None
    durata_text = None
    durata_min = None
    varsta = None
    limba = "Română"

    tag_by = soup.find(class_=re.compile(r"\bby\b", re.I))
    if tag_by:
        text_autor = tag_by.get_text(strip=True)
        autor = re.sub(r"^de\s*[:\-]?\s*", "", text_autor, flags=re.IGNORECASE).strip()

    if not autor:
        subtitlu = soup.find(class_=re.compile(r"subtit|author|subtitle", re.I))
        if subtitlu:
            text_sub = subtitlu.get_text(strip=True)
            autor = re.sub(r"^de\s*[:\-]?\s*", "", text_sub, flags=re.IGNORECASE).strip()

    for membru in echipa:
        rol = membru.get("role", "").lower()
        if "regie" in rol or "regizor" in rol:
            if not regizor:
                regizor = membru.get("name")
        elif "autor" in rol or "text" in rol or "dramaturgie" in rol:
            if not autor:
                autor = membru.get("name")

    text_complet = soup.get_text(" ", strip=True)

    m_durata = re.search(r'(?:durat[ă]|durată)\s*[:\-]?\s*([0-9]+\s*(?:h|ore?|min[a-
z]*)\s*(?:[0-9]+\s*min[a-z]*)?)', text_complet, re.IGNORECASE)
    if not m_durata:
        m_durata = re.search(r'(\b[0-9]{1,2}\s*h(?:\s*[0-9]{1,2}\s*min)?|\b[0-
9]{2,3}\s*min(?:ute)?)\b', text_complet, re.IGNORECASE)

    if m_durata:
        durata_text = m_durata.group(1).strip()
        ore_match = re.search(r'(\d+)\s*(?:h|ore?)', durata_text, re.I)
        min_match = re.search(r'(\d+)\s*min', durata_text, re.I)
        total = 0
```

```
        if ore_match:
            total += int(ore_match.group(1)) * 60
        if min_match:
            total += int(min_match.group(1))
        elif not ore_match and re.search(r'^\d+$', durata_text):
            total = int(durata_text)
        durata_min = total if total > 0 else None

    m_varsta =
re.search(r'(\b\d{1,2}\s*\+|\+\s*\d{2}|\b(?:peste|recomandat|vârsta|varsta)\s*\d{1,2}\s*ani|
\b\d{1,2}\s*ani\b)', text_complet, re.IGNORECASE)
    if m_varsta:
        gasit = m_varsta.group(1).strip()
        if gasit not in ["+1", "1+"]:
            varsta = gasit
        else:
            m_varsta_alt = re.search(r'(1[2468]\+|1[2468]\s*ani|\+1[2468])', text_complet)
            if m_varsta_alt:
                varsta = m_varsta_alt.group(1)

    return autor, regizor, durata_text, durata_min, varsta, limba

def porneste_crawler():
    url_spectacole = f"{url_baza}/spectacole/"
    if not permite_robots(url_spectacole):
        print("❌ Accesul este interzis prin robots.txt!")
        return

    print("Accesez pagina principală de spectacole...")
    try:
        pagina_principala = httpx.get(url_spectacole, headers=headere_browser,
follow_redirects=True, timeout=15.0)
        pagina_principala.raise_for_status()
        soup = BeautifulSoup(pagina_principala.text, "html.parser")
    except Exception as e:
        print(f"❌ Eroare la accesarea paginii principale: {e}")
        return

    toate_tagurile_a = soup.find_all("a", href=True)
    linkuri_spectacole = []
    for tag in toate_tagurile_a:
        href = tag.get("href")
        if "/spectacol/" in href and href not in linkuri_spectacole:
            linkuri_spectacole.append(href)

    print(f"Am găsit {len(linkuri_spectacole)} linkuri unice de spectacole!")
```

```
    luni = {
        "ian": 1, "ianuarie": 1, "feb": 2, "februarie": 2, "mar": 3, "martie": 3,
        "apr": 4, "aprilie": 4, "mai": 5, "iun": 6, "iunie": 6, "iul": 7, "iulie": 7,
        "aug": 8, "august": 8, "sep": 9, "sept": 9, "septembrie": 9,
        "oct": 10, "octombrie": 10, "noi": 11, "noiembrie": 11, "dec": 12, "decembrie": 12
    }

    for link in linkuri_spectacole:
        time.sleep(1.0)

        if not permite_robots(link):
            print(f"Omis conform robots.txt: {link}")
            continue

        try:
            pagina_spectacol = httpx.get(link, headers=headere_browser,
follow_redirects=True, timeout=15.0)
            pagina_spectacol.raise_for_status()
            soup_spect = BeautifulSoup(pagina_spectacol.text, "html.parser")

            titlu_tag = soup_spect.find("h1", class_="title")
            titlu = titlu_tag.text.strip() if titlu_tag else "Fără titlu"

            desc_tag = soup_spect.find("div", class_="content-info")
            descriere = desc_tag.text.strip() if desc_tag else ""

            afis_tag = soup_spect.find("div", class_="header-show")
            url_afis = None
            if afis_tag and afis_tag.get("style"):
                potrivire_url = re.search(r"url\(['\"]?(.*?)['\"]?\)",
afis_tag.get("style"))
                if potrivire_url:
                    url_afis = potrivire_url.group(1)

            cale_afis, hash_afis = descarca_afis(url_afis)

            container_distributie = soup_spect.find("div", class_="distributie")
            distributie = []
            echipa_artistica = []

            if container_distributie:
                linii = [l.strip() for l in
container_distributie.get_text(separator="\n").split("\n") if len(l.strip()) > 1]
                suntem_la_actori = False
                i = 0
```

```
                while i < len(linii) - 1:
                    stanga = linii[i]
                    dreapta = linii[i+1]
                    if "distributie" in stanga.lower() or "distribuție" in stanga.lower():
                        suntem_la_actori = True
                        i += 1
                        continue
                    if suntem_la_actori:
                        distributie.append({"character": stanga, "name": dreapta})
                    else:
                        echipa_artistica.append({"role": stanga, "name": dreapta})
                    i += 2

            json_distributie = json.dumps(distributie if distributie else [{"name": "Nu a
fost publicată"}], ensure_ascii=False)
            json_echipa = json.dumps(echipa_artistica if echipa_artistica else [{"role":
"Informație", "name": "Nepublicată"}], ensure_ascii=False)

            autor, regizor, durata_text, durata_min, varsta, limba =
extrage_metadate_spectacol(soup_spect, echipa_artistica)

            sql_show = """
                INSERT INTO shows
                (title, author, director, description, duration_minutes, duration_text,
age_recommendation, language,
                 cast_json, team_json, poster_storage, poster_path, poster_source_url,
poster_hash, source_url, last_seen_at)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, 'file', %s, %s, %s, %s,
NOW())
                ON DUPLICATE KEY UPDATE
                    title=VALUES(title),
                    author=VALUES(author),
                    director=VALUES(director),
                    description=VALUES(description),
                    duration_minutes=VALUES(duration_minutes),
                    duration_text=VALUES(duration_text),
                    age_recommendation=VALUES(age_recommendation),
                    language=VALUES(language),
                    cast_json=VALUES(cast_json),
                    team_json=VALUES(team_json),
                    poster_path=VALUES(poster_path),
                    poster_hash=VALUES(poster_hash),
                    last_seen_at=NOW()
            """
            cursor.execute(sql_show, (
                titlu, autor, regizor, descriere, durata_min, durata_text, varsta, limba,
```

```
                json_distributie, json_echipa, cale_afis, url_afis, hash_afis, link
            ))
            db.commit()

            cursor.execute("SELECT id FROM shows WHERE source_url = %s", (link,))
            id_spectacol = cursor.fetchone()["id"]

            elemente_calendar = soup_spect.find_all("div", class_="left")
            for element in elemente_calendar:
                date_tag = element.find("span", class_="date")
                time_tag = element.find("span", class_="time")

                rand_eveniment = element.find_parent("div",
class_=re.compile(r"item|row|program", re.I))
                if not rand_eveniment:
                    rand_eveniment = element.parent

                url_bilet = None
                if rand_eveniment:
                    tag_bilet = rand_eveniment.find("a",
href=re.compile(r"ticketsys|bilet|ticket|event", re.I))
                    if not tag_bilet:
                        tag_bilet = rand_eveniment.find("a", href=True)

                    if tag_bilet and tag_bilet.get("href"):
                        href_curat = tag_bilet["href"].strip()
                        if href_curat.startswith("http"):
                            url_bilet = href_curat
                        elif href_curat.startswith("/"):
                            url_bilet = f"{url_baza}{href_curat}"

                url_final_reprezentatie = url_bilet if url_bilet else link

                if date_tag and time_tag:
                    text_ora = time_tag.text.strip()
                    zi, luna, an = None, None, None

                    if url_bilet:
                        m_url = re.search(r'(\d{1,2})-(\d{1,2})-(\d{4})', url_bilet)
                        if m_url:
                            zi = int(m_url.group(1))
                            luna = int(m_url.group(2))
                            an = int(m_url.group(3))

                    if not (zi and luna and an):
```

```
                        text_complet_rand = rand_eveniment.get_text(" ", strip=True).lower()
if rand_eveniment else date_tag.text.strip().lower()

                        m_data = re.search(r'(\d{1,2})[-./](\d{1,2})[-./](\d{4})',
text_complet_rand)
                        if m_data:
                            zi = int(m_data.group(1))
                            luna = int(m_data.group(2))
                            an = int(m_data.group(3))
                        else:
                            m_zi = re.search(r'\b(\d{1,2})\b', date_tag.text)
                            if m_zi:
                                zi = int(m_zi.group(1))
                                mapare_luni = {
                                    'ian': 1, 'feb': 2, 'mar': 3, 'apr': 4,
                                    'mai': 5, 'iun': 6, 'iul': 7, 'aug': 8,
                                    'sep': 9, 'oct': 10, 'noi': 11, 'dec': 12
                                }
                                for nume_luna, nr in mapare_luni.items():
                                    if nume_luna in text_complet_rand:
                                        luna = nr
                                        break
                                an = datetime.now().year

                    if luna is not None and an is not None:
                        acum = datetime.now()
                        if acum.month >= 9 and luna < 9:
                            an = acum.year + 1

                    if zi and luna and an and ":" in text_ora:
                        ora, minut = text_ora.split(":")
                        data_finala = datetime(an, luna, zi, int(ora), int(minut))

                        sql_perf = """
                            INSERT INTO performances (show_id, start_at, venue, source_url,
last_seen_at)
                            VALUES (%s, %s, 'Teatrul Mic', %s, NOW())
                            ON DUPLICATE KEY UPDATE
                                venue=VALUES(venue),
                                source_url=VALUES(source_url),
                                last_seen_at=NOW()
                        """
                        cursor.execute(sql_perf, (id_spectacol, data_finala,
url_final_reprezentatie))
                        db.commit()
```

```
            print(f"✔ Salvat: {titlu} (Vârstă: {varsta or '-'}, Durată: {durata_text or '-
'})")

        except Exception as e:
            print(f"❌ Eroare la procesarea {link}: {e}")

if __name__ == "__main__":
    try:
        porneste_crawler()
    finally:
        cursor.close()
        db.close()
```

Anexa 3

```
import streamlit as st
import mysql.connector
import json
import os

st.set_page_config(page_title="Repertoriu Teatru", layout="wide")

@st.cache_resource
def init_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="abcd1234",
        database="theater_crawler"
    )

conn = init_connection()

def fetch_data(query, params=None):
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, params or ())
    return cursor.fetchall()

query_params = st.query_params
show_id = query_params.get("id")
page = query_params.get("page", "shows")

# --- RUTA 1: Program Reprezentații (/schedule) ---
if page == "schedule":
    st.title("🗓️ Program Reprezentații")
    if st.button("← Înapoi la spectacole"):
        st.query_params.clear()
```

```
        st.rerun()

    st.divider()

    reprezentatii = fetch_data("""
        SELECT p.start_at, p.venue, s.title
        FROM performances p
        JOIN shows s ON p.show_id = s.id
        ORDER BY p.start_at ASC
    """)

    if reprezentatii:
        for r in reprezentatii:
            r['Dată și Oră'] = r.pop('start_at').strftime("%d-%m-%Y %H:%M")
            r['Spectacol'] = r.pop('title')
            r['Sală / Locație'] = r.pop('venue')

        st.dataframe(reprezentatii, width="stretch", hide_index=True)
    else:
        st.info("Nu există reprezentații programate.")

# --- RUTA 2: Detalii Spectacol (/shows/{id}) ---
elif show_id:
    if st.button("← Înapoi la listă"):
        st.query_params.clear()
        st.rerun()

    spectacol_data = fetch_data("SELECT * FROM shows WHERE id = %s", (show_id,))

    if spectacol_data:
        s = spectacol_data[0]
        st.title(s['title'])

        # Link direct către spectacol pe site-ul oficial al teatrului
        if s.get('source_url'):
            st.markdown(f"🔗 **Pagină oficială teatru:**
[{s['source_url']}]({s['source_url']})")
            st.write("")

        col1, col2 = st.columns([1, 2])

        with col1:
            if s['poster_path'] and os.path.exists(s['poster_path']):
                st.image(s['poster_path'], width="stretch")
            else:
                st.info("Afiș indisponibil")
```

```
        with col2:
            if s.get('author'): st.write(f"**Autor:** {s['author']}")
            if s.get('director'): st.write(f"**Regizor:** {s['director']}")

            if s.get('duration_text'):
                st.write(f"**Durată:** {s['duration_text']}")
            elif s.get('duration_minutes'):
                st.write(f"**Durată:** {s['duration_minutes']} min")

            if s.get('age_recommendation'): st.write(f"**Recomandare de vârstă:**
{s['age_recommendation']}")
            if s.get('genre'): st.write(f"**Gen:** {s['genre']}")
            if s.get('language'): st.write(f"**Limbă:** {s['language']}")

            st.markdown("### Descriere")
            st.write(s['description'] if s['description'] else "Nicio descriere
disponibilă.")

        st.divider()

        col_cast, col_team = st.columns(2)

        with col_cast:
            st.subheader("🎭 Distribuție")
            cast = json.loads(s['cast_json']) if s['cast_json'] else []
            if cast:
                for actor in cast:
                    nume = actor.get('name', 'Necunoscut')
                    personaj = actor.get('character', '')
                    if personaj:
                        st.markdown(f"- **{nume}** (ca *{personaj}*)")
                    else:
                        st.markdown(f"- {nume}")
            else:
                st.write("Distribuție nepublicată.")

        with col_team:
            st.subheader("🎬 Echipă Artistică")
            team = json.loads(s['team_json']) if s['team_json'] else []
            if team:
                for membru in team:
                    rol = membru.get('role', 'Membru')
                    nume = membru.get('name', 'Necunoscut')
                    st.markdown(f"- **{rol}:** {nume}")
            else:
```

```
                st.write("Echipă nepublicată.")

        st.divider()

        st.subheader("🎟️ Reprezentații viitoare")
        # Extragem și source_url pentru linkul de cumpărare directă
        viitoare = fetch_data("SELECT start_at, venue, source_url FROM performances WHERE
show_id = %s AND start_at >= NOW() ORDER BY start_at ASC", (show_id,))
        if viitoare:
            for rep in viitoare:
                data_ora = rep['start_at'].strftime('%d %B %Y, %H:%M')
                locatie = rep['venue'] or 'Teatrul Mic'
                url_bilet = rep.get('source_url')

                if url_bilet and url_bilet.startswith("http"):
                    st.markdown(f"- 🗓️ **{data_ora}** | {locatie} ➡️ [🎟️ Cumpără Bilet /
Detalii Eveniment]({url_bilet})")
                else:
                    st.markdown(f"- 🗓️ **{data_ora}** | {locatie}")
        else:
            st.write("Nu există reprezentații viitoare programate pentru acest spectacol.")
    else:
        st.error("Spectacolul nu a fost găsit în baza de date.")

# --- RUTA 3: Lista Spectacolelor cu Bară de Căutare (Default) ---
else:
    st.title("🎭 Repertoriu Teatru")

    col_btn, col_search = st.columns([1, 2])
    with col_btn:
        if st.button("🗓️ Vezi Programul Complet"):
            st.query_params["page"] = "schedule"
            st.rerun()

    with col_search:
        # Bara de căutare dinamică obligatorie
        search_query = st.text_input("🔍 Caută spectacol după titlu...", "").strip()

    st.divider()

    if search_query:
        spectacole = fetch_data("SELECT id, title, author, director, poster_path,
duration_text FROM shows WHERE title LIKE %s", (f"%{search_query}%",))
    else:
```

```
        spectacole = fetch_data("SELECT id, title, author, director, poster_path,
duration_text FROM shows")

    if not spectacole:
        st.warning("Nu a fost găsit niciun spectacol conform căutării.")
    else:
        cols_per_row = 3
        rows = [spectacole[i:i + cols_per_row] for i in range(0, len(spectacole),
cols_per_row)]

        for row in rows:
            cols = st.columns(len(row))
            for idx, s in enumerate(row):
                with cols[idx]:
                    with st.container(border=True):
                        st.subheader(s['title'])

                        if s['poster_path'] and os.path.exists(s['poster_path']):
                            st.image(s['poster_path'], width="stretch")
                        else:
                            st.info("Afiș indisponibil")

                        if s.get('author'): st.caption(f"**Autor:** {s['author']}")
                        if s.get('director'): st.caption(f"**Regie:** {s['director']}")
                        if s.get('duration_text'): st.caption(f"⏱ **Durată:**
{s['duration_text']}")

                        next_perf = fetch_data("SELECT start_at FROM performances WHERE
show_id = %s AND start_at >= NOW() ORDER BY start_at ASC LIMIT 1", (s['id'],))
                        if next_perf:
                            st.success(f"➡ SOON {next_perf[0]['start_at'].strftime('%d-%m-%Y
%H:%M')}")
                        else:
                            st.caption("➡ SOON Nicio reprezentație programată")

                        if st.button("Vezi detalii", key=f"btn_{s['id']}", width="stretch"):
                            st.query_params["id"] = s['id']
                            st.rerun()
```